

อุปสรรคของลามะ

ลามะต้องการที่จะเดินทางข้ามที่ราบสูงแอนเดียน ลามะมีแผนที่ของที่ราบสูงในรูปแบบของกริดขนาด $N \times M$ แถวของแผนที่นั้นถูกเรียงลำดับด้วยตัวเลข 0 ถึง $N - 1$ นับจากบนลงล่าง และคอลัมน์ของแผนที่ถูกเรียงลำดับด้วยตัวเลข 0 ถึง $M - 1$ นับจากซ้ายไปขวา ช่องของแผนที่ในแถวที่ i คอลัมน์ที่ j ($0 \leq i < N, 0 \leq j < M$) ถูกแสดงโดยคู่อันดับ (i, j)

ลามะได้ทำการศึกษาภูมิอากาศของที่ราบสูงดังกล่าวและค้นพบว่า ทุก ๆ ช่องในแถวเดียวกันจะมีค่า **อุณหภูมิ** เท่ากัน และทุก ๆ ช่องในคอลัมน์เดียวกันจะมีค่า **ความชื้น** เท่ากัน ลามะได้ยื่นอาร์เรย์ของจำนวนเต็มให้กับคุณทั้งหมดสองอาร์เรย์ T และ H ซึ่งมีความยาว N และ M ตามลำดับ โดยที่ค่า $T[i]$ ($0 \leq i < N$) แสดงอุณหภูมิของช่องในแถวที่ i และค่า $H[j]$ ($0 \leq j < M$) แสดงค่าความชื้นของช่องในคอลัมน์ที่ j

ลามะได้ศึกษาเกี่ยวกับพันธุ์ไม้ในแถบที่ราบสูงและสังเกตเห็นว่าช่องที่ (i, j) **ปราศจากพันธุ์ไม้ใด ๆ** เมื่อและต่อเมื่อค่าอุณหภูมิในช่องนั้นสูงกว่าค่าความชื้น หรือกล่าวอย่างเป็นทางการได้ว่า $T[i] > H[j]$

ลามะสามารถเดินทางข้ามพื้นที่ราบสูงนี้ได้ตาม **เส้นทางที่ใช้ได้** โดยเส้นทางที่ใช้ได้เป็นลำดับของช่องที่ไม่ซ้ำกันในแผนที่ ที่มีคุณสมบัติต่อไปนี้

- แต่ละคู่ของช่องที่เรียงต่อกันในเส้นทางนั้นจะต้องเป็นช่องที่อยู่ติดกัน
- ทุกช่องที่อยู่ในเส้นทางนั้นจะต้องปราศจากพันธุ์ไม้ใด ๆ

งานของคุณคือการตอบคำถามทั้งหมด Q ข้อ แต่ละคำถามจะถูกระบุด้วยจำนวนเต็มสี่ตัว ได้แก่ L, R, S และ D คุณจะต้องการหาว่ามีเส้นทางที่ใช้ได้หรือไม่ โดยเส้นทางดังกล่าวจะต้อง

- เริ่มต้นที่ช่อง $(0, S)$ และสิ้นสุดที่ช่อง $(0, D)$
- ทุกช่องในเส้นทางนั้นจะต้องอยู่ระหว่างคอลัมน์ L และคอลัมน์ R รวมหัวท้าย

รับประกันว่าช่อง $(0, S)$ และช่อง $(0, D)$ นั้นจะปราศจากพันธุ์ไม้ใด ๆ

รายละเอียดการเขียนโปรแกรม

ฟังก์ชันแรกที่คุณต้องเขียนคือ

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : อาร์เรย์ความยาว N แสดงค่าอุณหภูมิในแต่ละแถว
- H : อาร์เรย์ความยาว M แสดงค่าความชื้นในแต่ละคอลัมน์
- ฟังก์ชันนี้จะถูกเรียกเพียงหนึ่งครั้งเท่านั้นสำหรับแต่ละกรณีทดสอบ ก่อนที่จะเรียกใช้ฟังก์ชัน `can_reach`

ฟังก์ชันที่สองที่คุณต้องเขียนคือ

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : จำนวนเต็มที่ใช้ระบุคำถาม
- ฟังก์ชันนี้จะถูกเรียกใช้ Q ครั้งสำหรับแต่ละกรณีทดสอบ

ฟังก์ชันนี้ต้องคืนค่าเป็น `true` เมื่อและต่อเมื่อมีเส้นทางที่ใช้ได้จากช่อง $(0, S)$ ไปยังช่อง $(0, D)$ โดยที่ทุกช่องบนเส้นทางนั้นจะต้องอยู่ระหว่างคอลัมน์ L และคอลัมน์ R รวมหัวท้าย

เงื่อนไข

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ สำหรับแต่ละ i ที่ $0 \leq i < N$
- $0 \leq H[j] \leq 10^9$ สำหรับแต่ละ j ที่ $0 \leq j < M$
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- ช่อง $(0, S)$ และช่อง $(0, D)$ ปราศจากพันธมิตรไม่ว่าใด ๆ

ปัญหาย่อย

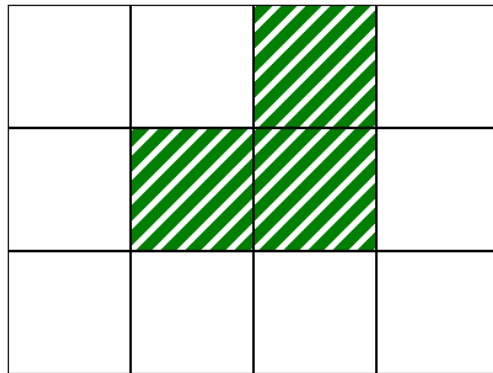
ปัญหาย่อย	คะแนน	เงื่อนไขเพิ่มเติม
1	10	$L = 0, R = M - 1$ สำหรับแต่ละคำถาม และ $N = 1$
2	14	$L = 0, R = M - 1$ สำหรับแต่ละคำถาม และ $T[i - 1] \leq T[i]$ สำหรับแต่ละ i ที่ $1 \leq i < N$
3	13	$L = 0, R = M - 1$ สำหรับแต่ละคำถาม และ $N = 3, T = [2, 1, 3]$
4	21	$L = 0, R = M - 1$ สำหรับแต่ละคำถาม และ $Q \leq 10$
5	25	$L = 0, R = M - 1$ สำหรับแต่ละคำถาม
6	17	ไม่มีเงื่อนไขเพิ่มเติม

ตัวอย่าง

พิจารณาการเรียกใช้ฟังก์ชันต่อไปนี้:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

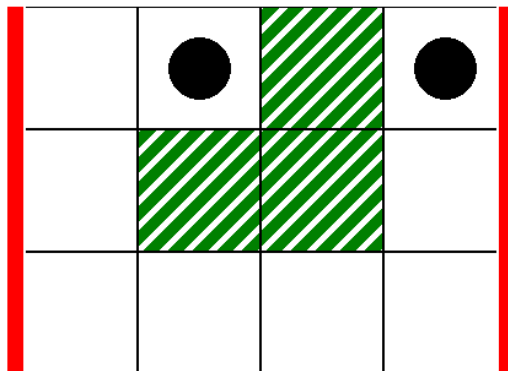
ซึ่งสอดคล้องกับแผนที่ในรูปต่อไปนี้ โดยช่องสีขาวคือช่องที่ปราศจากพันธมิตรใด ๆ



สำหรับคำถามแรก พิจารณาการเรียกฟังก์ชันดังนี้:

```
can_reach(0, 3, 1, 3)
```

ซึ่งแสดงถึงสถานการณ์ในรูปด้านล่าง โดยที่เส้นหนาแนวตั้งบ่งบอกถึงช่วงของคอลัมน์ $L = 0$ ถึงคอลัมน์ $R = 3$ และจุดสีดำแสดงถึงช่องเริ่มต้นและช่องสิ้นสุด



ในกรณีนี้ ลามะสามารถเดินทางจากช่อง (0, 1) ไปยังช่อง (0, 3) ผ่านเส้นทางที่ใช้ได้ต่อไปนี้:

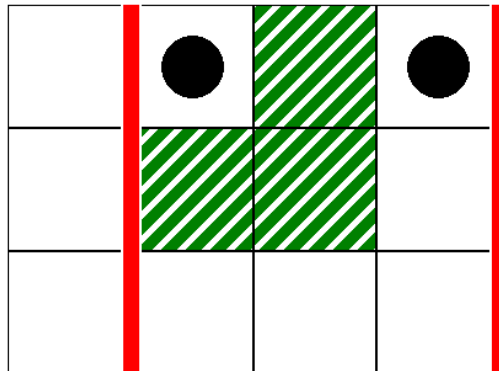
(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)

ดังนั้น ฟังก์ชันจะต้องคืนค่าเป็น `true`

สำหรับคำถามที่สอง พิจารณาการเรียกฟังก์ชันดังนี้:

```
can_reach(1, 3, 1, 3)
```

ซึ่งแสดงถึงสถานการณ์ในรูปแบบด้านล่าง:



ในกรณีนี้ ไม่มีเส้นทางที่ใช้ได้จากช่อง (0,1) ไปยังช่อง (0,3) โดยที่แต่ละช่องอยู่ระหว่างคอลัมน์ 1 และคอลัมน์ 3 รวมหัวท้าย ดังนั้น ฟังก์ชันจะต้องคืนค่าเป็น `false`

เกร็ดเดอร์ตัวอย่าง

รูปแบบข้อมูลนำเข้า:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

ในที่นี้ $L[k], R[k], S[k]$ และ $D[k]$ ($0 \leq k < Q$) ระบุถึงค่าพารามิเตอร์สำหรับการเรียกฟังก์ชัน `can_reach`

รูปแบบข้อมูลส่งออก:

```
A[0]
A[1]
...
A[Q-1]
```

ในที่นี้ $A[k]$ ($0 \leq k < Q$) มีค่าเป็น 1 ถ้าฟังก์ชัน `can_reach(L[k], R[k], S[k], D[k])` ส่งคืนค่า `true` นอกนั้นจากนั้น ค่าจะเป็น 0